

Debugging avanzado gdb

Herramientas del sistema

Antonio Espín Herranz

Contenidos

- Uso avanzado de gdb: breakpoints condicionales, watchpoints y TUI.
- Análisis de fallos con core dumps.
- Trazado con strace/ltrace.
- Análisis de memoria con Valgrind y sanitizers.
- Diagnóstico de concurrencia y herramientas del sistema para detectar cuellos de botella, corrupción de memoria y problemas de sincronización.

Preparación del programa

- Compilar con símbolos de depuración:

```
gcc -std=c11 -Wall -Wextra -Wpedantic \  
-g -O0 programa.c -o programa
```

- Parámetros:

-g: incorpora información de depuración.

-O0: evita que las optimizaciones dificulten el seguimiento inicial.

Para investigar errores dependientes de optimización también conviene probar con -Og, -O2 o -O3.

Iniciar GDB

- `gdb ./programa`
- Con argumentos:
 - `gdb --args ./programa argumento1 argumento2`

Uso avanzado de gdb

- breakpoints condicionales
- watchpoints
- TUI

Breakpoints Básicos

- Detenerse al entrar en una función.
 - break **main**
 - break **procesar_datos**
- Detenerse en una línea:
 - break programa.c:42
- Listar breakpoints:
 - info breakpoints
- Desactivar, activar o eliminar:
 - disable 1
 - enable 1
 - delete 1

Breakpoints condicionales

- Permiten detener el programa solamente cuando se cumple una expresión.
 - `break procesar_datos if longitud > 1024`
- **Ejemplos:**
 - `break servidor.c:80 if client_fd < 0`
 - `break worker if thread_id == 3`
 - `break procesar if buffer == NULL`
 - `break bucle.c:25 if i == 999`

Breakpoints

- Añadir una condición a un breakpoint existente:
 - `condition 2 counter > 1000`
- Eliminar una condición:
 - `condition 2`
- **GDB** evalúa la expresión cada vez que alcanza el breakpoint y solo detiene la ejecución cuando es verdadera.

Breakpoints

- Ignorar las primeras activaciones
 - ignore 2 100
 - Ignora las primeras 100 activaciones del breakpoint número 2.

Watchpoints

- Un **watchpoint** detiene el programa cuando cambia el valor de una variable o expresión.
- Es especialmente útil para localizar quién corrompe un dato.
- Detenerse cuando cambia una variable:
 - watch counter
- Detenerse cuando cambia el valor apuntado:
 - watch *pointer

Watchpoints

- Vigilar un **campo**:
 - `watch connection->state`
- Vigilar un **array**:
 - `watch buffer[50]`
- **Condición** adicional:
 - `watch counter if counter > 100`

Tipos de Watchpoints

- **watch** variable
 - Detiene al escribir y cambiar el valor.
- **rwatch** variable
 - Detiene al leer.
- **awatch** variable
 - Detiene al leer o escribir

Navegación

- Ejecutar el programa: **run**
- Continuar: **continue**
- Ejecutar una línea entrando en las funciones: **step**
- Ejecutar una línea sin entrar en las funciones: **next**
- Terminar la función actual: **finish**
- Continuar hasta una línea: **until 80**

Inspección

- **Mostrar una variable:**

- print counter
- print *pointer
- print structure.field

- **Formatos:**

- print /x value
- print /d value
- print /t value
- print /c character

- **Para threads:**

- info threads
- thread apply all bt

- ***Fallos en programas concurrentes.***

- **Mostrar variables locales y parámetros:**

- info locals
- info args

- **Mostrar la pila de llamadas:**

- **backtrace**
- Abreviada: **bt**

- **Mostrar un frame:**

- **frame 2**

- **Examinar memoria:**

- x/16bx buffer
- x/8wx pointer
- x/s text

GDB TUI

TUI ofrece una interfaz visual dentro del terminal.

Iniciar directamente: **`gdb -tui ./programa`**

Activar / desactivar dentro de GDB:
Ctrl+X A

Vistas principales:

- layout src
- layout asm
- layout regs
- layout split

- **src**: código fuente.
- **asm**: ensamblador.
- **regs**: registros.
- **split**: código fuente y ensamblador.

Actualizar la pantalla: Ctrl+L

Análisis de fallos con core dumps

- Un **core dump** es una captura del estado de un proceso en el momento en que termina debido a una señal fatal, por ejemplo: **SIGSEGV**.
- Puede incluir memoria, registros, pila de llamadas, hilos y valores de variables, y puede analizarse posteriormente con GDB.
- Un **core dump** puede producirse con desreferencia de un puntero nulo y normalmente termina con SIGSEGV

Ejemplo: crash.c

```
#include <stdio.h>

static void process_value(int *value){
    printf("Valor: %d\n", *value);
}

int main(void){
    int *pointer = NULL;
    process_value(pointer);
    return 0;
}
```

COMPILACION:

```
gcc -std=c11 -Wall -Wextra -Wpedantic \
-g -O0 crash.c -o crash
```

Core dumps

- Consultar el límite actual: **ulimit -c**
- Si devuelve 0, están deshabilitados para esa sesión.
 - Para activarlos: **ulimit -c Unlimited**
- Verificar: **ulimit -c**
- ***El cambio afecta a la terminal actual y a los procesos iniciados desde ella.***

Core dumps

- Ejecutar crash: **./crash**
- Resultado esperado: **Segmentation fault (core dumped)**
- Consultar información del **Kernel**:
 - **cat /proc/sys/kernel/core_pattern**
- Si muestra un nombre como **core**:
 - El fichero puede generarse en el directorio de trabajo.

Analizar un core local con GDB

Se necesita:

- El ejecutable exacto que produjo el fallo.
 - Su core dump.
 - Preferiblemente, símbolos de depuración.
 - **gdb ./crash ./core**
-
- Dentro de **GDB**:
 - backtrace o bt
-
- Salida:
 - Program terminated with signal SIGSEGV
 - #0 process_value at crash.c:5
 - #1 main at crash.c:12

Generar un core de un proceso activo

- GDB puede generar una captura sin finalizar el proceso:
 - **gcore PID**
 - Suele crear: **core.PID**
 - Análisis: **gdb ./programa core.PID**

Es útil para analizar:

- deadlocks;
- procesos bloqueados;
- servidores que dejaron de responder;
- estados difíciles de reproducir.

Limitaciones y Seguridad

Un core dump puede contener:

- contraseñas;
 - tokens;
 - claves criptográficas;
 - datos personales;
 - buffers de red.
-
- No debe publicarse ni compartirse sin revisarlo. Además, el core debe analizarse con el ejecutable y las bibliotecas correspondientes a la versión que produjo el fallo.

coredumpctl

- Si el sistema utiliza un gestor compatible, se pueden listar los fallos: **coredumpctl list**
- Mostrar información del último: **coredumpctl info**
- Abrir el último core directamente con GDB: **coredumpctl debug**
- Exportar un core: **coredumpctl dump PID --output=core.dump**
- Después: **gdb ./programa core.dump**

Trazado con strace/ltrace

Ambas herramientas permiten analizar un programa sin modificar su código:

- **strace**: muestra llamadas al sistema y señales.
- **ltrace**: muestra llamadas a bibliotecas dinámicas.
- **Instalación:**
 - `sudo apt update`
 - `sudo apt install strace ltrace`

strace: llamadas al sistema

- **strace** intercepta las **llamadas** realizadas en la frontera **entre** el **programa** y el **kernel**, mostrando sus argumentos, valores de retorno y errores.
 - `openat()`
 - `read()`
 - `write()`
 - `socket()`
 - `connect()`
 - `accept()`
 - `recvfrom()`
 - `sendto()`
 - `mmap()`
 - `futex()`

Uso básico:

strace ./programa

Guardar la salida:

strace -o trace.log ./programa

strace: interpretar una línea

- `openat(AT_FDCWD, "config.txt", O_RDONLY) = 3`

Significa:

- Se intentó abrir config.txt.
 - Se abrió en modo lectura.
 - El descriptor obtenido es 3.
-
- Con **error**: `openat(AT_FDCWD, "config.txt", O_RDONLY) = -1 ENOENT`
 - ENOENT significa que el fichero o directorio no existe.
 - Los errores aparecen con retorno **-1**, código **errno** y **descripción**

strace: Filtrar llamadas

- Solo **operaciones** relacionadas con **ficheros**:
 - `strace -e trace=file ./programa`
- Solo **red**:
 - `strace -e trace=network ./servidor`
- Sólo **procesos**:
 - `strace -e trace=process ./programa`
- Sólo **señales**:
 - `strace -e trace=signal ./programa`
- **Llamadas concretas**:
 - `strace -e trace=openat,read,write ./programa`

strace: Seguir hilos y procesos hijo

- `strace -f ./programa`

La opción **-f** es importante para aplicaciones con:

- `pthread_create();`
 - `fork();`
 - servidores concurrentes.
-
- Guardar cada proceso en un fichero separado:
 - `strace -ff -o trace ./programa`
 - Genera ficheros del tipo:
 - `trace.1234`
 - `trace.1235`
 - `trace.1236`

strace: conectarse a un proceso activo

- Primero se obtiene el PID: **pgrep** programa
- **sudo strace -p PID**
- **sudo strace -p 12345** (*suponemos el PID del proceso: 12345*)
- **Detener el trazado** con **Ctrl+C** no debería finalizar el proceso al que se conectó strace.

Tiempos y Rendimientos

- Añadir marcas temporales: **strace -tt ./programa**
- Mostrar cuanto dura cada llamada: **strace -T ./programa**
- Combinar ambas: **strace -tt -T ./programa**
- Generar un resumen estadístico: **strace -c ./programa**
- El modo **-c** resume número de llamadas, errores y tiempo empleado por cada llamada al sistema.

Ejemplo

- `sudo strace -f -p PID \`
- `-e trace=network,read,write,futex`
- Puede mostrar que el proceso está esperando en:
 - `accept(...)`
 - `recvfrom(...)`
 - `epoll_wait(...)`
 - `futex(...)`

Esto ayuda a distinguir entre:

- espera de conexiones;
- espera de datos;
- espera de eventos;
- espera de sincronización.

ltrace: llamadas a librerías

- **ltrace** intercepta **llamadas** realizadas a **bibliotecas dinámicas** y puede mostrar sus **parámetros** y valores de **retorno**.
- También puede mostrar llamadas al sistema mediante opciones específicas.
- **Ejemplos:**
 - malloc()
 - free()
 - printf()
 - strlen()
 - strcmp()
 - fopen()
 - memcpy()

ltrace

- **Uso básico: ltrace ./programa**

- Guardar el resultado: **ltrace -o ltrace.log ./programa**

- **Filtrar funciones:**

- Solo llamadas a **malloc**:
 - ltrace -e **malloc** ./programa
- Varias funciones:
 - ltrace -e malloc+free+realloc ./programa
- Funciones relacionadas con cadenas:
 - ltrace -e strlen+strcmp+strcpy ./programa
- Seguir procesos hijo:
 - ltrace -f ./programa

- Conectarse a un proceso:
 - sudo ltrace -p PID

- Resumen estadístico:
 - ltrace -c ./programa

- La opción -c cuenta llamadas y tiempos por función de biblioteca.

Comparativa

Herramienta	Observa	Ejemplos
strace	Interacción con el kernel	openat, read, socket, connect, futex
ltrace	Llamadas a bibliotecas dinámicas	malloc, free, printf, strlen
GDB	Estado interno y ejecución	Variables, memoria, pila y breakpoints
Valgrind	Errores dinámicos	Fugas, accesos inválidos y uso tras liberación

Uso

- No encuentra un fichero → **strace**
- Problema de conexión o socket → **strace**
- Proceso bloqueado en una llamada del sistema → **strace**
- Quiero observar funciones de biblioteca → **ltrace**
- Necesito inspeccionar variables y flujo interno → **GDB**
- Sospecho corrupción de memoria → **Valgrind o sanitizers**

A tener en cuenta

- ***strace y ltrace alteran los tiempos de ejecución, por lo que pueden modificar o esconder fallos sensibles a temporización, especialmente en programas concurrentes.***

Análisis de memoria con Valgrind y sanitizers

- Estas herramientas permiten localizar errores de memoria durante la ejecución:
 - accesos fuera de límites;
 - uso después de free();
 - doble liberación;
 - memoria no inicializada;
 - fugas de memoria;
 - comportamiento indefinido.

Programa de prueba

```
#include <stdlib.h>
int main(void){
    int *values = malloc(4 * sizeof(*values));
    if (values == NULL) {
        return EXIT_FAILURE;
    }
    values[4] = 100;
    free(values);
    values[0] = 200;
    return EXIT_SUCCESS;
}
```

Contiene:

- escritura fuera del bloque;
- uso después de liberar memoria.

Valgrind Memcheck

- **Memcheck** detecta accesos inválidos, valores no inicializados, liberaciones incorrectas, bloques solapados y fugas de memoria.
- **Instalación:**
 - `sudo apt update`
 - `sudo apt install valgrind`
- **Compilación:**
 - `gcc -std=c11 -Wall -Wextra -Wpedantic -g -O0 memoria.c -o memoria`

Valgrind Memcheck

- **Ejecución básica:**

valgrind ./memoria

- **Análisis completo:**

```
valgrind \  
  --tool=memcheck \  
  --leak-check=full \  
  --show-leak-kinds=all \  
  --track-origins=yes \  
  ./memoria
```

- **Mensajes habituales:**

- Invalid read
- Invalid write
- Invalid free
- Use of uninitialised value
- definitely lost

- **Categorías de fugas:**

- **definitely lost:** no queda ningún puntero válido.
- **indirectly lost:** memoria dependiente de otro bloque perdido.
- **possibly lost:** queda un puntero potencialmente interior.
- **still reachable:** existe un puntero válido, pero no se liberó.

AddressSanitizer

ASan instrumenta el programa durante la compilación y detecta principalmente:

- heap-buffer-overflow;
- stack-buffer-overflow;
- use-after-free;
- dobles liberaciones;
- accesos inválidos.

Se activa mediante **-fsanitize=address**

AddressSanitizer

- **Compilación:**

```
gcc -std=c11 -Wall -Wextra -Wpedantic \  
-g -O1 \  
-fsanitize=address \  
-fno-omit-frame-pointer \  
memoria.c -o memoria_asan
```

- **Ejecución:**

```
./memoria_asan
```

ASan suele detener la ejecución al detectar el primer error grave. Debe corregirse y volver a ejecutar el programa.

UndefinedBehaviorSanitizer

UBSan detecta determinadas operaciones con comportamiento indefinido:

- desbordamiento aritmético con signo;
- división entera por cero;
- desplazamientos inválidos;
- accesos desalineados;
- determinados índices y conversiones inválidos.

```
gcc -std=c11 -Wall -Wextra -Wpedantic \  
-g -O1 \  
-fsanitize=undefined \  
-fno-omit-frame-pointer \  
programa.c -o programa_ubsan
```

ASan y UBSan combinados

- Configuración recomendada durante el desarrollo:
 - Compilación:
 - `gcc -std=c11 -Wall -Wextra -Wpedantic \`
 - `-g -O1 \`
 - `-fsanitize=address,undefined \`
 - `-fno-omit-frame-pointer \`
 - `programa.c -o programa_debug`
 - Ejecución:
 - `./programa_debug`

Comparación

Característica	Valgrind	Sanitizers
Requiere recompilar	No necesariamente	Sí
Velocidad	Más lento	Generalmente más rápido
Fugas	Memcheck	LeakSanitizer
Accesos inválidos	Memcheck	ASan
Memoria no inicializada	Muy útil	ASan no está orientado a ello
Comportamiento indefinido	Limitado	UBSan
Uso recomendado	Análisis detallado final	Desarrollo y pruebas continuas

Flujo recomendado

Compilar con warnings



Ejecutar ASan y UBSan



Corregir los errores



Ejecutar Valgrind Memcheck



Comprobar fugas y accesos inválidos

Diagnóstico de concurrencia y herramientas del sistema

- cuellos de botella
- corrupción de memoria
- problemas de sincronización

ThreadSanitizer: Race conditions

- **ThreadSanitizer** detecta accesos simultáneos incompatibles a memoria compartida.

- **Compilación:**

- gcc -std=c11 -g -O1 \
- -fsanitize=thread \
- -fno-omit-frame-pointer \
- programa.c -o programa_tsan \
- -pthread

- **Ejecución:**

- ./programa_tsan

El informe identifica:

- variable o dirección compartida;
 - hilos implicados;
 - lectura o escritura conflictiva;
 - líneas donde se produjeron los accesos.
-
- ThreadSanitizer requiere recompilar e instrumentar el programa.
-
- ***TSan no debe combinarse en el mismo binario con AddressSanitizer.***

Helgrind: sincronización POSIX

- **Helgrind** analiza programas que utilizan pthreads.
- **Compilación:**
 - `gcc -std=c11 -g -O0 \`
 - `programa.c -o programa \`
 - `-pthread`
- **Ejecución:**
 - `valgrind --tool=helgrind ./programa`

Puede detectar:

- carreras de datos (races conditions);
 - órdenes inconsistentes de mutexes;
 - posibles deadlocks;
 - desbloques incorrectos;
 - destrucción de mutexes activos;
 - uso incorrecto de variables de condición.
-
- Helgrind está específicamente orientado a errores de sincronización con primitivas POSIX

DRD

- **DRD** es otra herramienta de Valgrind para programas multihilo:

Puede detectar:

- carreras de datos;
 - contención de locks;
 - errores en el uso de pthreads;
 - secciones críticas excesivamente largas.
-
- **Helgrind** y **DRD** pueden utilizarse como comprobaciones complementarias, no simultáneamente.

Analizar un bloqueo GDB

- Conectarse al proceso: **gdb -p PID**
- Dentro del **GDB**:
 - info threads
 - thread apply all backtrace
- Seleccionar un hilo:
 - thread 3
- Examinar su **pila**:
 - backtrace
 - frame 0
 - info locals
- Un deadlock suele mostrar varios hilos detenidos en:
 - pthread_mutex_lock()
 - pthread_cond_wait()
 - futex()

Analizar un bloqueo GDB

- Para desconectarse sin terminar el proceso:
 - detach
 - quit

Captura un proceso bloqueado

- Puede generarse un core sin finalizar el proceso: **gcore PID**
- **gdb ./programa core.PID**
- Dentro de GDB: **thread apply all bt full**

Uso de CPU por hilo

- top: top **-H** -p PID

La opción **-H** muestra los hilos individualmente.

Interpretación básica:

- Un hilo con CPU elevada puede estar en un bucle intensivo o livelock.
- Muchos hilos activos pueden indicar exceso de paralelismo.
- CPU baja con aplicación bloqueada puede indicar espera de locks o E/S.

ps

- `ps -L -p PID -o pid,tid,psr,stat,pcpu,comm`

Muestra:

- PID del proceso;
- TID del hilo;
- CPU donde se ejecuta;
- estado;
- porcentaje de CPU.

Cuellos de botella con perf

- Estadísticas generales:
 - `perf stat ./programa`

Permite observar eventos como:

- ciclos de CPU;
- instrucciones;
- cambios de contexto;
- fallos de página;
- fallos de caché.

- Proceso activo
 - `sudo perf stat -p PID`

- Funciones que consumen CPU
 - `sudo perf top -p PID`

- Grabar un perfil:
 - `perf record -g ./programa`

- Analizarlo:
 - `perf report`

Corrupción de memoria

- Para accesos fuera de límites o memoria liberada:

Compilación:

```
gcc -g -O1 \  
-fsanitize=address,undefined \  
-fno-omit-frame-pointer \  
programa.c -o programa_asan \  
-pthread
```

Ejecución: ./programa_asan

```
valgrind \  
--tool=memcheck \  
--leak-check=full \  
--track-origins=yes \  
./programa
```

- ***Memcheck detecta accesos inválidos, valores no inicializados, liberaciones incorrectas y fugas.***

Mensaje del Kernel

- Consultar mensajes recientes:
`sudo dmesg --level=err,warn`
- Buscar problemas concretos:
`sudo dmesg | grep -Ei \`
`"segfault|oom|killed process|general protection"`

Puede ayudar a localizar:

- procesos terminados por falta de memoria;
- fallos de segmentación registrados por el kernel;
- errores de hardware o controladores.

- **Aplicación bloqueada**
 - GDB y thread apply all bt
- **CPU elevada**
 - top -H y perf
- **Carrera de datos**
 - ThreadSanitizer
- **Deadlock u orden incorrecto de locks**
 - Helgrind
- **Corrupción de memoria**
 - ASan, UBSan y Memcheck
- **Espera en kernel o E/S**
 - strace
- **Terminación inesperada**
 - core dump y dmesg

Flujo recomendado

Ninguna herramienta detecta todos los problemas.
Lo habitual es **combinar** información de GDB, sanitizers, Valgrind, strace, perf y los registros del sistema.